

实验四 长度扩展攻击

课程名称：信息安全

课程代码：SOFT130018.01

任课教师：李景涛

实验课助教：杨鹏宇 24210240377@m.fudan.edu.cn

张安琪 24212010048@m.fudan.edu.cn

实验目的

1. 了解Merkle-Damgård 结构;
2. 了解长度扩展攻击的原理;
3. 进行长度扩展攻击;

实验原理

长度扩展攻击是一种针对某些哈希函数的攻击，特别是针对基于Merkle-Damgård 结构的哈希函数，如 MD5、SHA-1 和 SHA-256。这些哈希函数可以表示为 $H(k||m)$ ，其中 H 表示哈希函数， k 表示密钥， m 表示消息。长度扩展攻击利用了这些哈希函数的工作原理，使攻击者能够在不知道密钥的情况下，扩展原消息内容并为其生成一个有效的新哈希。这对于基于哈希的消息认证码（HMAC）或数字签名等场景可能带来安全隐患。

背景：哈希函数和 Merkle-Damgård 结构

哈希函数是一种将任意长度的输入（消息）映射为固定长度输出的算法，常用于数据完整性校验和密码学。许多哈希函数（如 MD5、SHA-1 和 SHA-256）使用了Merkle-Damgård 结构，分块处理消息。

Merkle-Damgård 结构的工作方式如下：

- 将输入消息按固定大小分块（例如 512 位）。
- 使用压缩函数 f 、初始向量 s_1 、第一个消息块 b_1 ，计算 $s_2 = f(s_1, b_1)$ 。
- 计算 $s_{i+1} = f(s_i, b_i)$ ，直到处理完所有消息块。
- 最终得到的压缩函数值是整个消息的哈希值。

长度扩展攻击原理

由于哈希函数在处理消息时通过块操作，攻击者可以利用这种方式生成新消息的哈希值，而不需要知道原始消息内容。具体步骤如下：

1. 已知信息：

- 攻击者知道消息 M ($M = k||m$) 的长度和它的哈希值 $H(M)$ ，并且知道使用的哈希函数。
- 攻击者不知道密钥 k ，因此攻击者不知道消息 M 。

2. 扩展攻击：

- 哈希函数通常需要在最后一块中填充消息（padding），以保证消息长度符合要求。
- 攻击者可以通过添加 *suffix* 伪造一个扩展消息 M' ，得到 $M' = M + padding + suffix$ 。

- 基于 Merkle-Damgård 结构，攻击者可以通过哈希值 $H(M)$ ，计算得到 $H(M')$ ，而无需知道原始消息 M 。

3.结果:

- 攻击者生成的新消息 M' 将具有合法的哈希值，因为它有效地扩展了原始消息的哈希计算。
- 这个新哈希值看起来像是由 $M + padding + suffix$ 计算得出的，欺骗了接收方，使其认为该消息和哈希值是合法的。

实验环境

运行 Windows 操作系统的计算机，具有 C/C++语言编译环境（原则上操作系统和编程语言 环境不限）。

实验内容

4.1 使用 python 验证长度扩展

为了实验长度扩展攻击的思想，我们将使用MD5哈希函数的Python实现，（SHA-1和SHA-256同样容易受到长度扩展的攻击）。

1. 下载并导入 pymd5 模块

- 下载lab4中的pymd5模块。

2. 计算原始消息的 MD5 哈希

- 原始消息 M 为 "Use HMAC, not hashes"。
- 计算其MD5哈希并打印结果。

3. 计算扩展后消息的 MD5 哈希

- 附加新suffix:"Good advice" 。
- 在原消息哈希值的基础上计算添加了扩展消息后的新哈希值 H_1 。

4. MD5 的填充机制

MD5处理消息时，会对消息进行填充，并确保填充后的bit长度为512bit的倍数，然后将填充后的结果分成长度为512bit的块。填充的方式是先添加一个比特1，然后添加必要数量的比特0，最后附加消息的bit长度（64位小端序表示）。

- 使用 pymd5 模块中的 `padding()` 函数为原始消息 M 计算所需的填充。

```
pad = padding(len(M)*8)
```

5. 验证结果

最后，可以验证扩展后的哈希 H_1 是否等于伪造的消息 $M' = M + padding(len(M)*8) + suffix$ 的哈希值 H_2 。

- 计算并输出 `M + padding(len(M)*8) + suffix` 的哈希 H_2 。
- 验证 H_1 是否等于 H_2 。

【要求】

文件名: `verify_len_ext.py`

最终输出:

1. 原始消息 M 的MD5哈希。
2. 扩展消息的MD5哈希 H_1 。
3. 伪造消息 M' 的哈希值 H_2 。
4. H_1 与 H_2 是否相等。

4.2 长度扩展攻击

长度扩展攻击是一种针对某些哈希函数（如MD5、SHA-1和SHA-256）的攻击方式，攻击者可以利用已知的哈希值和消息长度，构造出新的哈希值，而无需知道原始消息的内容。在下面的例子中，攻击者将在没有获得用户权限的情况下进行非法操作，给数据造成破坏。

有一个API接口，可以通过客户端使用特定格式的URL执行用户操作，一个示例URL及其查询参数的说明如下：

```
https://cs.fdu.edu.cn/lab4/api?
token=0f8349d0a9aa1da4d53749fdff404d28&user=stu&command1=ListFiles&command2=NoOp
```

token: 通过MD5计算得出，格式为 $H(k||m)$ ， k 表示用户的密钥（长度为8个字节）， m 表示从user开始到URL结束的部分（即 `user=stu&command1=ListFiles&command2=NoOp` 部分）。

user: 用户的用户名。

command1 和 command2: API命令。

攻击者的目标是利用长度扩展攻击，在不知道用户密钥 k 的情况下，构造一个新的URL，使其有效地执行 `DeleteAllFiles` 命令（即添加参数 `command3=DeleteAllFiles`）。

需要：

1. 解析URL：使用Python的 `urlparse` 模块解析输入示例URL，提取出相关参数。
2. 计算扩展URL后的新哈希：利用原消息的token和要添加的参数，计算出新的token。
3. 构造新URL：根据新token、padding（使用Python的 `urllib` 模块中的 `quote()` 函数来编码URL中的非ASCII字符）和suffix构造伪造URL，使其能通过哈希校验。

【要求】

文件名: `len_ext_attack.py`

最终输出：可用于攻击的新URL，并写在实验报告中。

实验提交

截至日期：2025年5月11日23:59前

提交清单：

1. 实验报告pdf格式，文件名格式：学号_姓名_lab4.pdf；
2. 项目源代码：

文件1: verify_len_ext.py,
文件2: len_ext_attack.py;

3. 录制视频，展示代码结构、演示运行结果等。

提交方式：将提交清单中所有文件打包成一个压缩文件（文件名：学号_姓名_lab4），在elearining上提交。

评分标准

标准						
源代码可编译运行	√	√	√	√	√	
源代码风格良好	√		√			
程序运行结果正确	√	√	√	√		
源代码可编译运行	√	√			√	√
得分	100	90-99	80-89	60-79	40-59	20-39

注：

1. “源代码风格良好”指的是有必要的注释、合适的缩进，变量和函数命名便于理解；
2. 若出现两位同学报告或代码完全一致的情况，则双方本次实验成绩均为 0；
3. 若源码与程序无法正常运行，则成绩不高于 60 分；
4. 其他情况酌情给分。